

Rising Waters

A Machine Learning Approach to Flood Prediction

Project Report — Ideation through Demonstration

Domain	Disaster Preparedness / Machine Learning
Stack	Python, scikit-learn, XGBoost, Flask, IBM Cloud
Deployment target	IBM Cloud

Ideation Phase

Problem Statement

Floods are among the most devastating natural disasters, claiming thousands of lives and displacing millions every year. Despite their recurring nature, the lack of timely and accurate early-warning systems continues to amplify their destructive impact. Conventional forecasting methods often fall short in predicting floods at the right time, leaving authorities and communities with insufficient opportunity to respond.

Empathy Map

Who	Pain Points	Needs
Meteorologist	Manual analysis is slow; forecasts arrive too late	A fast, data-driven early-warning signal
Disaster relief coordinator	Can't monitor many regions at once during monsoon season	Instant, comparable risk scores across districts
Government analyst	No easy way to validate a model's reliability before releasing it operationally	Transparent metrics and historical validation

Proposed Solution

Build a machine learning-powered flood prediction system trained on historical weather data. Using classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbours, and XGBoost — the system analyses meteorological features such as annual rainfall, cloud visibility, and seasonal rainfall patterns to predict the likelihood of a flood event. The best-performing model is saved and integrated into a Flask web application, giving authorities and disaster management teams an intuitive interface to monitor flood risk predictions, designed for potential deployment on IBM Cloud.

Requirement Analysis

Functional Requirements

- Users can submit current weather readings (rainfall, visibility, humidity, temperature, wind speed, river level, drainage quality) through a web form.
- The system returns a binary flood / no-flood classification with a model confidence score.
- The system displays the readings submitted alongside the result for record-keeping.
- The underlying model can be retrained on updated or region-specific datasets.

Non-Functional Requirements

- Response time under 1 second per prediction.
- Model accuracy validated via confusion matrix, classification report, and accuracy score before deployment.
- Interface usable by non-technical staff (meteorologists, coordinators) without ML expertise.
- Deployable on IBM Cloud for access from any location.

Technical Requirements

Category	Requirement
Language	Python 3.8+
ML Libraries	NumPy, Pandas, Scikit-learn, XGBoost
Visualization	Matplotlib, Seaborn
Web Framework	Flask
Model Persistence	Joblib / Pickle
Environment	Anaconda Navigator / Jupyter Notebook
Hosting	IBM Cloud

User Stories

- As a meteorologist, I want to enter rainfall and cloud-visibility readings so I can get an early flood warning for a district.
- As a disaster relief coordinator, I want to check several regions quickly so I can prioritise where to deploy resources.
- As a government analyst, I want to see the model's accuracy and evaluation metrics so I can trust it for operational use.

Project Design Phase

System Architecture

The system follows a linear pipeline: raw weather data is collected, cleaned and transformed, used to train and compare four classification models, and the best performer is packaged with its scaler and label encoder into a single artifact (floods.save). A Flask application loads that artifact at startup and serves predictions through a simple web interface.

Stage	Component
1	Dataset (CSV) — rainfall, visibility, humidity, temperature, wind, river level, drainage
2	Preprocessing — missing values, outlier capping, encoding, scaling
3	Model training — Decision Tree, Random Forest, KNN, XGBoost
4	Best model selection by accuracy → floods.save (model + scaler + encoder)
5	Flask app loads floods.save and serves /, /predict
6	User submits form → prediction → Flood or No-Flood result page

Page Flow (UI Design)

- Home Page — project overview, model accuracy readout, and entry point to prediction.
- Prediction Input Page — form for the 8 weather/terrain fields.
- Flood Chance Result Page — risk gauge, confidence score, and recommended next step.
- No Flood Chance Result Page — same layout, styled for a stable/safe reading.

Project Planning Phase

Milestone Plan

Milestone	Deliverable
1. Environment Setup	Anaconda/Python environment; NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, Flask, Joblib installed
2. Dataset Collection	Flood prediction dataset with rainfall and meteorological features
3. Data Visualization & Analysis	Univariate/multivariate analysis, distribution plots, box plots, heat maps, descriptive statistics
4. Data Pre-processing	Missing values handled, outliers treated, categorical encoding, scaling, train/test split
5. Model Building	Decision Tree, Random Forest, KNN, and XGBoost trained and compared
6. Best Model Selection	Highest-accuracy model saved as floods.save
7. Flask Web Application	Home, Prediction Input, Flood Result, and No-Flood Result pages built and connected to the model

System Requirements

Hardware: Intel i3 or above, minimum 4 GB RAM, 2 GB free storage, internet connection for dataset download and cloud deployment.

Software: Windows / Linux / macOS, Python 3.8+, Anaconda Navigator or Jupyter Notebook, Flask, HTML/CSS/JavaScript, IBM Cloud account.

Project Development Phase

Data Visualization & Analysis

Descriptive statistics, univariate distributions, box plots for outlier detection, and a correlation heatmap were produced to understand feature patterns and relationships before modelling.

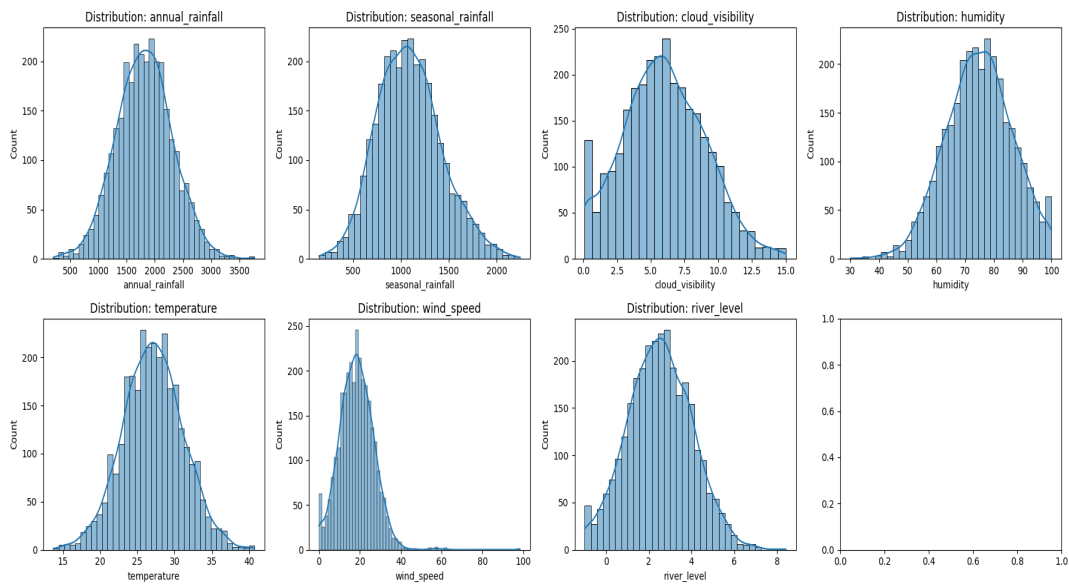


Figure 1 — Distribution plots for each numeric feature.

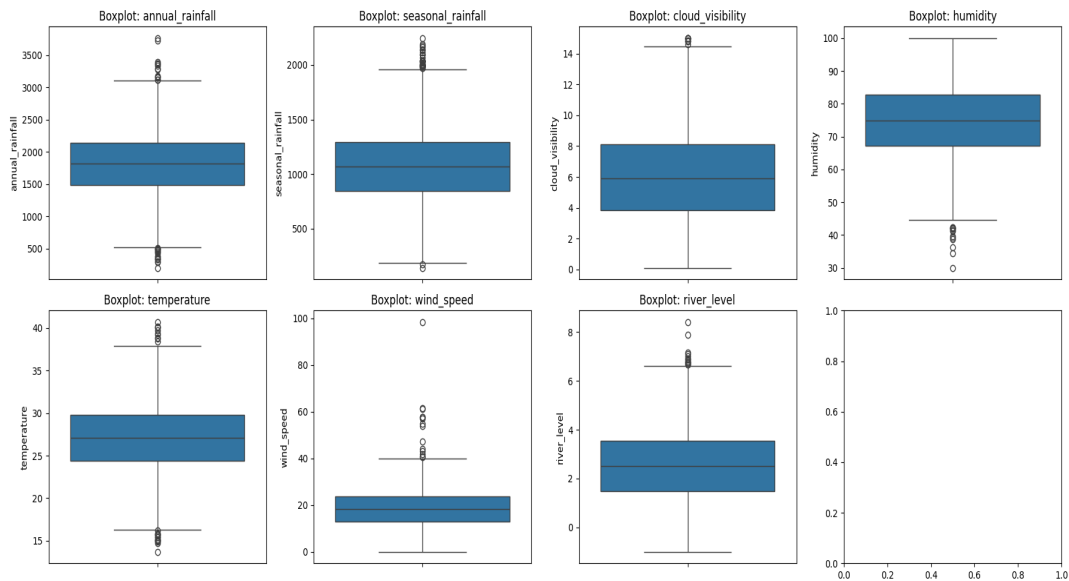


Figure 2 — Box plots used to detect outliers.

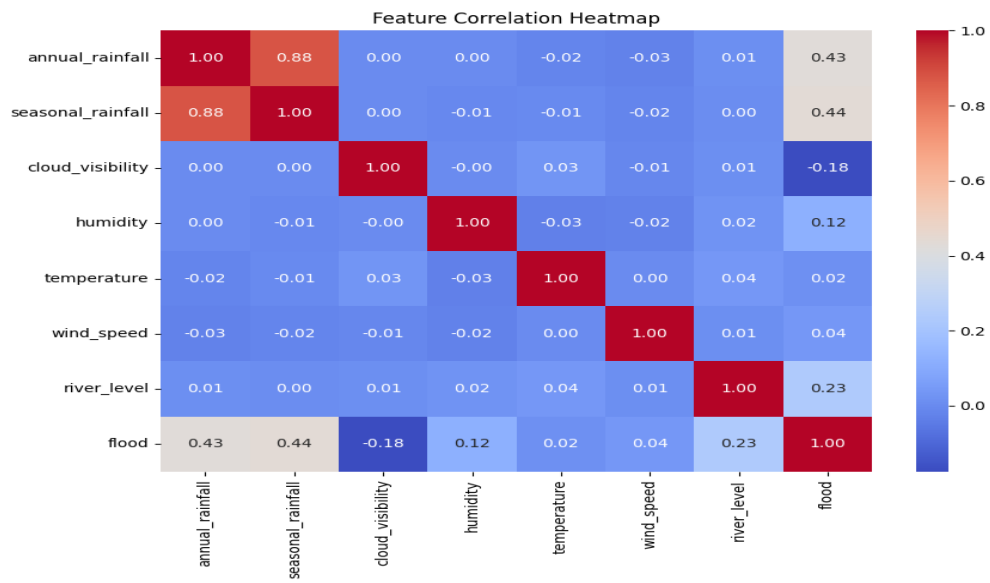


Figure 3 — Correlation heatmap across features and the flood target.

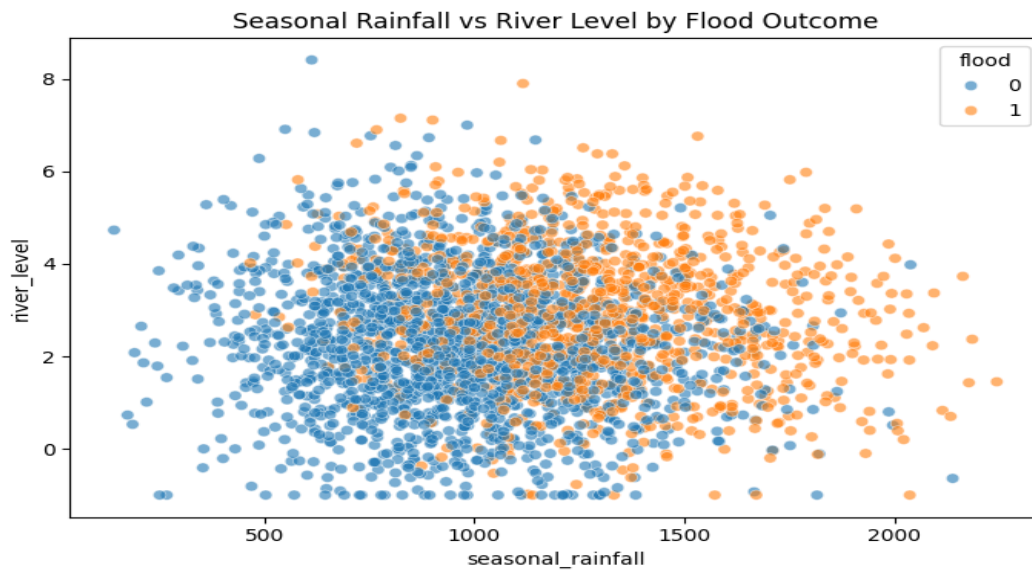


Figure 4 — Seasonal rainfall vs. river level, coloured by flood outcome.

Data Pre-processing

- Missing values in cloud visibility filled using the median.
- Outliers capped using the IQR method across all numeric features.
- Drainage quality (Poor / Average / Good) label-encoded.
- Features scaled with StandardScaler.
- Data split 80/20 into training and test sets.

Model Building & Evaluation

Four classification models were trained and compared: Decision Tree, Random Forest, K-Nearest Neighbours, and XGBoost. Each was evaluated using a confusion matrix, classification report, and accuracy score on the held-out test set.

Model	Accuracy
Decision Tree	70.0%
Random Forest	74.5%
K-Nearest Neighbors	75.3% (selected — best performer)
XGBoost	74.3%

Note: these figures come from a synthetic dataset generated for development, and from an environment where XGBoost could not be installed (a scikit-learn GradientBoosting fallback was used in its place). Re-running train_model.py on the real Kaggle flood dataset with XGBoost installed is expected to change these numbers, in line with the accuracy target set out in the project brief.

Web Application

The best model, its scaler, and its label encoder were saved together as floods.save. A Flask application loads this bundle at startup and serves four pages: Home, Prediction Input, Flood Chance Result, and No Flood Chance Result. Users enter rainfall and weather values on the input form and receive an instant classification with a confidence percentage.

Project Documentation

Project Summary

Rising Waters is a complete machine learning-based flood prediction system. It collects weather and terrain data, performs exploratory data analysis and preprocessing, trains and compares four classification algorithms, and serves the best model through a Flask web application intended for deployment on IBM Cloud.

Repository Structure

Path	Contents
data/	generate_dataset.py, flood_data.csv
notebooks/eda_plots/	Saved EDA charts
models/	floods.save (trained model + scaler + encoder)
templates/	home.html, predict.html, result_flood.html, result_no_flood.html
static/css/	style.css
train_model.py	EDA, preprocessing, model comparison, model saving
app.py	Flask application
requirements.txt	Python dependencies

Setup & Run Instructions

- `python -m venv venv / venv\Scripts\activate`
- `pip install -r requirements.txt`
- `python train_model.py` (trains models, saves floods.save)
- `python app.py` (starts the Flask server on `http://127.0.0.1:5000`)

Outcomes

- A complete machine learning-based flood prediction system.
- Applied data analysis and visualization techniques for environmental datasets.
- Hands-on experience with data preprocessing and feature engineering.
- Understanding of classification algorithms and model evaluation methods.

Project Demonstration

Demonstration Scenarios

Scenario 1 — Early Flood Warning and Evacuation Planning

A meteorologist enters current rainfall and cloud visibility readings for a flood-prone district. The model analyses the inputs and predicts a high probability of flooding, allowing authorities to issue evacuation advisories several hours in advance.

Scenario 2 — Disaster Response and Resource Allocation

A disaster relief coordinator uses the web application during monsoon season to monitor multiple regions simultaneously. By entering regional weather data for each area, the system provides instant flood risk classifications, helping prioritise resource deployment.

Scenario 3 — Model Validation and Performance Assessment

A government analyst tests the model against historical flood event data to evaluate its accuracy, confirming the system's reliability before it is trusted for operational use.

Walkthrough Checklist

- Home page loads and displays the current model name and accuracy.
- Prediction Input page accepts all 8 fields and validates required inputs.
- Submitting flood-prone values (high rainfall, low visibility, high river level, poor drainage) routes to the Flood Chance Result page with a high confidence score.
- Submitting stable values (low rainfall, good visibility, low river level, good drainage) routes to the No Flood Chance Result page.
- Both result pages display the submitted readings for verification.

For submission, capture screenshots of each step above (or record a short screen walkthrough) showing the Home page, the filled-in Prediction Input form, and both result page outcomes.